An AI Agent Pipeline with Smarter Problem Splitting for Pasqal's Quantum Computers, Applied to Portfolio Selection

Research plan

Hamzah Azzam, St. Olaf College
Supervisor: Dr. Michael Haydock

October 2026

Abstract

Pasqal's quantum computers solve one family of optimization problems directly: choosing the largest set of items in which no two items conflict. A problem can be placed on the machine directly only if its conflicts can be represented as distances between atoms, and most real business problems do not meet this condition. Pasqal's researchers showed that converting such problems to fit the machine requires too many atoms and can badly degrade the answers, and proposed an alternative: divide the problem into pieces that fit, solve each piece on the machine, and combine the results [1]. In their method, a fixed rule decides which piece to take next. We will replace this rule with rules learned by several types of AI, including neural networks and reinforcement learning, and test whether they produce better answers or require fewer runs on the quantum computer. The learned selector will be part of an AI agent pipeline, in which a user describes a problem in plain language with a spreadsheet of data, and the agent formulates the problem, runs it, and reports the answer. We will develop and test the system on portfolio selection: choosing the largest set of stocks whose prices do not move together, which gives a diversified portfolio.

1. Background

1.1 How Pasqal's machine solves optimization problems

Pasqal's machine holds individual atoms in a flat pattern. Two atoms placed close together cannot both be switched on at the same time [4]. If each atom represents an item and closeness represents a conflict, the machine searches for the largest group of items with no conflicts. Like other approaches to this problem, it returns good answers that are not guaranteed to be the best possible. Many practical problems have this form, for example choosing cell tower sites that do not interfere with each other, or assigning work shifts that do not overlap.

1.2 Problems that do not fit the machine

This works only when every conflict in a problem can be represented by two atoms being close together. Most real problems do not have this property. Two work shifts conflict because of a scheduling rule, not because of a distance. Such problems can be converted to fit the machine by adding helper atoms [5]. However, Pasqal's researchers proved, under a standard assumption from complexity theory, that any such conversion needs a number of atoms that grows at least with the square of the problem size [1]. They also showed that the conversion can destroy answer quality: an answer that is nearly the best on the converted problem can be no better than a random guess on the original one.

1.3 Solving the problem in pieces

Pasqal's researchers proposed solving the problem in pieces instead [1]. Their method repeats the following steps:

1. Take a piece of the problem that fits on one of the machine's ready-made grids of atoms.
2. Solve that piece on the machine.

3. Remove the piece, together with every item that conflicts with it.
4. Repeat on what is left until nothing remains, then combine the answers from all pieces into one final answer.

To avoid depending on a single sequence of choices, the method explores several alternatives at once. In each round it tries four candidate pieces, keeps the two best answers for each, and carries forward only the ten most promising partial solutions.

On random test problems with up to 500 items, this method found better answers than a simple greedy algorithm, which repeatedly picks the item with the fewest conflicts. The authors also report how far their answers fall from the best answers found by two strong classical solvers, CPLEX and simulated annealing, and do not claim to beat them. Results on Pasqal's hardware agreed with simulation. The same method has since been applied to molecular docking problems in drug discovery [2].

1.4 Where this project contributes

Step 1 is decided by a fixed rule. It starts from a randomly chosen item, places it at the center of the grid, and grows the piece outward: it repeatedly takes an item already placed and tries to add that item's unplaced neighbors, in an order set by a simple ranking such as how many connections each has. Each new item goes into the first nearby position on the grid that keeps the problem's conflicts correct. Each candidate piece is built without considering how it will affect later rounds. Trying several candidates partly makes up for this, but only by spending more runs on the machine. Because each round permanently removes part of the problem, early choices affect every round that follows, and the authors note that obtaining large pieces is essential for fast solving, since many small pieces become a bottleneck.

Practice problems can be generated in any number, and the result of each choice can be measured. This makes the decision well suited to machine learning. We have not found any published work that applies learning to it.

2. The business problem: portfolio selection

2.1 Choosing stocks that do not move together

An investor who wants a diversified portfolio needs stocks whose prices do not rise and fall together. This can be written as the problem Pasqal's machine solves. Each stock is an item, and two stocks conflict if their daily returns over a recent period are highly correlated. The largest set of stocks with no conflicts is then a portfolio in which no two holdings move closely together. This representation, known as a market graph, is established in finance research [7]. It has been solved with a quantum-inspired machine to build portfolios from about 1,700 Japanese stocks [8], and recently on a trapped-ion quantum computer for stock indices of up to 225 stocks, using a different method that combines quantum runs with classical simplification steps [9]. To our knowledge it has not been solved on Pasqal's machines by splitting it into pieces.

2.2 Why it suits this project

- The conflicts come from price data, not from physical distance, so the problem does not fit the machine directly and must be solved in pieces. This is the case the project addresses.
- The S&P 500 contains about 500 stocks, the same size as the largest problems Pasqal tested [1]. Pasqal tested randomly generated problems, while market graphs have a very different structure, with a few stocks that conflict with many others and clusters of stocks from the same industry. We will therefore run Pasqal's current rule on our own market problems as the baseline, rather

than compare with their published numbers.

- Recomputing the conflicts over a moving window of many years of daily prices produces thousands of realistic problems, which provides the training data the learned selectors need.
- The correlation level that defines a conflict controls how many pairs of stocks conflict, and so how dense each problem is.

2.3 Testing in calm and crisis markets

During market crises, such as 2008 and March 2020, stocks tend to move together, so many more pairs conflict and the problems become much denser. We expect this to make the splitting decision harder, and we will test whether it does. We will train selectors on calm periods and test them on crisis periods to check whether their advantage holds when conditions change. We will also measure how the selected portfolios perform afterwards, for example their volatility in the following months. This version of the problem chooses which stocks to hold, not how much of each to buy, so it serves as the selection step of portfolio construction. Weighting stocks by expected return is a possible extension.

3. What we will build



3.1 The AI agent

A user gives the agent a problem described in plain language, together with a spreadsheet of data, for example a request for a diversified portfolio along with a spreadsheet of daily stock prices. The agent identifies which items conflict, checks whether the problem fits on the machine as given or must be divided into pieces, writes the program for Pasqal's machine using Pasqal's software [6], and converts the result back into an answer in the user's terms.

Pasqal's researchers recently published an agent that takes a published paper, a patent, or a plain-language request and runs the resulting experiment on Pasqal's machines, including optimization case studies [3]. They found that expert review remained essential, because the agent made mistakes that only a specialist caught. Our agent differs in two ways: it formulates business optimization problems from a user's own data, and it includes a learned step for splitting problems that do not fit the machine.

3.2 The learned selector

The learned selector is the main research contribution. It examines the remaining part of the problem and decides which piece to take next and where to place each item on the grid. We will build and compare several types of AI for this task, from simpler to more complex:

- A learned ranking: a small model that learns better scores for choosing the next item, replacing the fixed ranking in Pasqal's rule.
- A graph neural network: a neural network designed to read problems shaped as networks of connected items. It is first trained to copy Pasqal's current rule, then trained on examples of choices that led to better final answers.
- Reinforcement learning: a model that improves through trial and error, rewarded for better final

answers and for fewer runs on the machine.

Each approach will be compared with Pasqal's current rule, including its search over several candidate pieces, and with the others on the same problems. A simple greedy algorithm and strong classical solvers will serve as reference points. Comparing several approaches shows whether learning helps, and how much model complexity is needed to get the benefit.

4. Research questions

Q1. Do learned selectors find better answers than Pasqal's current rule on the same problems?

Q2. Can they reach answers of the same quality with fewer runs on the quantum computer?

Q3. Which type of AI works best, and does the added complexity of reinforcement learning pay off?

Q4. Does a selector trained on calm market periods still perform well during market crises, when stocks move together more and the problems become denser?

Q5. Can the agent reliably take a portfolio request from a plain description and price data to a verified result on the quantum computer?

References

[1] P. Cazals, A. Sorondo, V. Onofre, C. Dalyac, W. da Silva Coelho, and V. Vitale, *Quantum optimization on Rydberg atom arrays with arbitrary connectivity: Gadgets limitations and a heuristic approach*, arXiv:2508.06130 (2025).

[2] M. Garrigues, V. Onofre, W. Coelho, and S. Acheche, *A scalable heuristic for molecular docking on neutral-atom quantum processors*, arXiv:2508.18147 (2025).

[3] C. Dalyac, A. Dauphin, L. Henriet, and C. Jurczak, *Lowering the implementation barrier of neutral-atom quantum computing with agentic workflows*, arXiv:2607.25834 (2026).

[4] S. Ebadi et al., *Quantum optimization of maximum independent set using Rydberg atom arrays*, Science **376**, 1209 (2022).

[5] M.-T. Nguyen et al., *Quantum optimization with arbitrary connectivity using Rydberg atom arrays*, PRX Quantum **4**, 010316 (2023).

[6] H. Silvério et al., *Pulser: An open-source package for the design of pulse sequences in programmable neutral-atom arrays*, Quantum **6**, 629 (2022).

[7] V. Boginski, S. Butenko, and P. M. Pardalos, *Statistical analysis of financial networks*, Computational Statistics & Data Analysis **48**, 431 (2005).

[8] R. Hidaka, Y. Hamakawa, J. Nakayama, and K. Tatsumura, *Correlation-diversified portfolio construction by finding maximum independent set in large-scale market graph*, arXiv:2308.04769 (2023).

[9] R. Yalovetzky, M. J. A. Schuetz, et al., *Quantum-informed portfolio selection: An end-to-end pipeline validated on trapped-ion hardware with real market data*, arXiv:2607.01037 (2026).